WSVIPER: A Tool for WebSocket Analysis

Tejas Mysore Harish
Dept. of CSE
PES University
Bengaluru, Karnataka
tejasharish1234@gmail.com

Puskar Mishra
Dept. of CSE
PES University
Bengaluru, Karnataka
mishrapuskar2004@gmail.com

Prasad Honnavalli
Dept. of CSE
PES University
Bengaluru, Karnataka
prasadhb@pes.edu

***Abstract*—WebSocket is a technology that allows a client and a server to communicate in real-time, using a single TCP connection. WebSockets are a significantly faster means of communication compared to HTTP, enabling fast real-time communication by keeping the TCP connection open indefinitely. However, this increase in communication speed comes at the cost of minimal security. The lack of proper security can lead to exposure of sensitive data, unauthorized access, and many more problems. This research discusses the detection and testing of WebSockets in real-world web applications. WebSocket endpoints of websites are obtained through the process of crawling, and a comprehensive set of 90 attacks are run on the endpoints, including tests on handshaking, session management, origin and authentication checks, encryption, Denial of Service (DOS), resource management, cross-origin attacks, and protocol fuzzing attacks. By running these tests, we obtain a complete report on all vulnerabilities for a given WebSocket that can be used by a security team to make their system more secure.**

***Index Terms*—WebSocket, Vulnerabilities, Security, Real websites, WS/WSS**

I. Introduction

What is a WebSocket?

WebSocket is a full-duplex protocol for communication, as standardized by IETF in RFC 6455 [1]. The system operates on a single, long-lived TCP connection, enabling real-time, bidirectional communication between servers and clients. Rather than using a request-response cycle for each data exchange, WebSocket provides an open TCP channel through which messages can be sent and received at any moment, without delay. Thus, it is suitable for time-critical web applications. Real-time capabilities, such as live chats, stock updates, multiplayer gaming, and collaborative editing, are made possible by WebSocket communication in modern web applications [2].

The initiation of a WebSocket connection is via an HTTP request. The client sends an initial HTTP handshake and then upgrades to WebSocket by sending a connection upgrade request header. After the channel is initiated, it persists until both ends terminate it explicitly. The upgraded method enhances HTTP's intrinsic security features, but introduces some new issues which can be exploited without adequate security precautions [3], [4].

How is a WebSocket different from HTTP?

Although both WebSocket and HTTP utilize TCP and commonly operate over ports 80 or 443, their communication models differ significantly. HTTP follows a rigid request–response model in which all communication is initiated by the client and terminated after the server sends a response. In contrast, WebSocket provides persistent, event-driven communication in which both endpoints can transmit messages at any time once the connection has been established [5], [6].

HTTP is a stateless protocol and includes several mature security mechanisms, such as CORS, CSRF protection, caching controls, and standardized authentication methods. WebSocket connections, however, are stateful and long-lived, and are typically established dynamically through client-side JavaScript. As WebSocket primarily focuses on enabling low-latency real-time communication, it does not inherently provide many of the security controls available in HTTP. Consequently, while WebSocket offers significant performance advantages for dynamic applications, its flexibility requires developers to manually implement appropriate security best practices.

II. Problem Statement and Objectives

Although there is a rise in the use of WebSockets in contemporary web designs, security research for their use are still largely constrained to HTTP. The majority of present vulnerability detectors and web application firewalls (WAFs) concentrate on HTTP traffic and lack significant support for analysing the WebSocket protocol which creates blind spots in enterprise threat modelling. Additionally, most developers rely on the channel's security following a successful WebSocket handshake to prevent potential abuse by attackers.

The lack of strict validation and access control measures can weaken the security of WebSockets for developers. Since a WebSocket connection remains continually open, an attacker can use it to gain unauthorized access to the server and exchange messages with it without requiring further authentication [7] [8], [9].

The tool built in this research is an automated medium for testing WebSocket security with focus on real-world websites and how they operate. The system is designed to:

- Discover WebSocket endpoints by simulating the browser's behaviour.
- Implement 90 protocol-compliant attack scenarios from 9 vulnerability classes.
- Use heatmaps, bar charts, and detailed summaries to report results.

III. Literature Survey

A. *WebSocket Security Analysis : J Erkkilä (2012)*

This is the first important research into WebSockets. It focuses on analyzing the security concerns associated with WebSockets and emphasizes that, while the technology improves connectivity, it does not inherently address security issues. The author discusses potential solutions and recommend best practices for secure deployment, including the need for stronger security features at the browser level. They also point out that WebSocket technology is still evolving and lacks a comprehensive, standardized security model. Overall, the paper concludes that ensuring WebSocket security requires coordinated efforts from browser vendors, protocol designers, and service developers.

B. *Security Testing of WebSockets: H. Kuosmanen (2016)*

This work studies security vulnerabilities in WebSocket implementations and highlights that many traditional web vulnerabilities extend to WebSockets, albeit with protocol-specific variations. It emphasizes the lack of mature and comprehensive tools for WebSocket security testing compared to HTTP-based applications. To address this gap, the author develops a semi-manual testing tool capable of identifying common vulnerabilities, though it requires significant expertise from the tester. The study also notes that existing literature does not provide a complete catalog of WebSocket vulnerabilities, relying instead on practitioner knowledge and real-world insights. Overall, the work demonstrates the feasibility of building a dedicated WebSocket testing tool but lacks automation and comprehensive coverage.

C. *Security aspects of full-duplex web interactions and WebSockets: Y. Fu and M. Garcia-Valls (2023)*

This study have highlighted the growing use of WebSockets for enabling real-time, low-latency communication in modern web applications. While the protocol improves efficiency compared to traditional HTTP, it introduces several security challenges. The authors have identified the primary vulnerabilities as cross-site WebSocket hijacking, improper authentication, and lack of input validation. Many existing approaches focus on securing communication using encryption and token-based authentication mechanisms. However, these solutions often fail to address scalability and real-time threat detection in large-scale systems. Therefore, there is a need for more robust and automated security frameworks for WebSocket-based applications.

The reviewed literature highlights that while WebSocket technology enables efficient real-time communication, it introduces significant security challenges that are not fully addressed by existing standards or tools. Prior works identify common vulnerabilities such as authentication issues, cross-site WebSocket hijacking, while emphasizing the lack of comprehensive and automated testing frameworks. Existing solutions are often limited in scalability, coverage, and require professional understanding.

WSVIPER addresses the identified gaps by proposing a comprehensive and automated framework for WebSocket endpoint vulnerability analysis. It combines browser automation, intelligent crawling, and a comprehensive analysis of WebSockets to provide an understanding of the various vulnerabilities in modern WebSocket infrastructure, with the goal of secure management in the near future. Unlike existing approaches, the proposed system offers broader coverage, minimal manual effort, and improved effectiveness in detecting real-world WebSocket vulnerabilities.

IV. Implementation

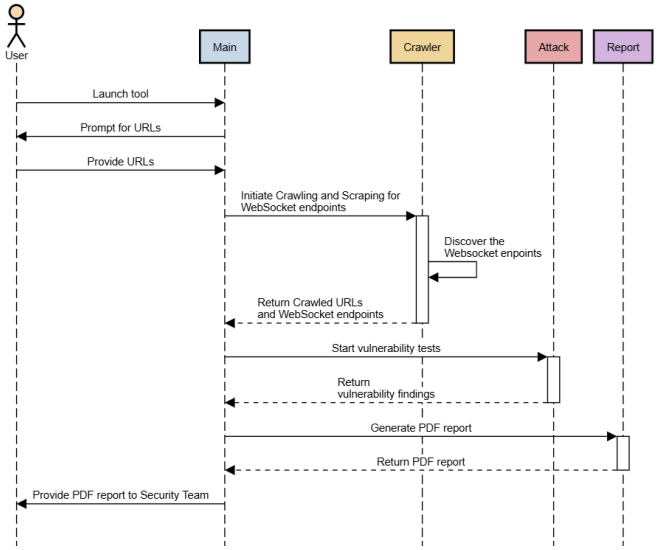

Fig. 1. Sequence Diagram of WSVIPER Implementation

Explanation for sequence diagram

Fig. 1 shows the sequence diagram which illustrates the workflow of the tool, beginning via the main file. The user/ security team can either manually enter a website URL or upload a list of WebSockets to be tested(this skips the crawling phase). Once the input is received, the main module begins execution of the process of crawling.

The crawler module uses the Playwright library to parse through the site. It scans through all URLs and retrieves WebSocket endpoints and relevant metadata, and returns them to the main module. The process through which this is done is explained in detail in Stage 1. If no WebSockets are found, the tool prompts the user to enter the WebSocket to be tested.

The discovered WebSocket URLs are then forwarded to the attack module for security evaluation. This module performs over 90 distinct vulnerability tests targeting common WebSocket security flaws. These include malformed handshake requests, missing or expired authentication tokens, spoofed origin headers, improper session handling, and more. The attacks are explained in Stage 2.

Finally, all test results are passed to the report generator module. This module compiles the findings into a structured,

visually informative PDF report. The report includes graphs such as bar charts, pie charts, and heatmaps, which illustrate various findings in a pictorial manner for a quick understanding on problems, but also proceeds to go in depth, explaining the issues in detail.

Stage 1: Finding the WebSocket Endpoints

A. Browser Automation with Playwright: Microsoft's Playwright, an open-source browser automation library, is utilized for real-time page analysis [10]. Through APIs that emulate user interactions, JavaScript programming, and network traffic interception, Playwright provides complete control over browser behaviour through Chromium, a headless browser (it does not have a GUI).

All HTTP and WebSocket traffic, including dynamically generated resources and delayed WebSocket initializations triggered by AJAX or script evaluation, is monitored during web browsing.

Playwright employs various methods of stealth, such as:

- Randomization of user-agent strings.
- The fabrication of browser fingerprinting APIs.
- Disabling automation detection flags.

With these features, the crawler can bypass bot detection mechanisms used by many modern websites, continuing to provide complete endpoint discovery. Through Playwright, the system can:

- Load complex web applications.
- Interact with dynamic elements.
- Observe WebSocket handshake requests.
- Obtain real-time WebSocket URLs that are hidden or deferred.

B. Web Crawler and Scraper: WSVIPER is designed to crawl websites to identify resources that are accessible through programming methods. The crawler utilises browser automation to simulate real user interactions, rather than relying solely on static HTML. By rendering web content dynamically, it can uncover resources that static analysis might overlook.

In addition, a scraping module has been integrated to extract valuable information from both the page and network responses. This includes embedded JavaScript, JSON payloads, inline HTML, and similar data, all of which enhance the system's capability for thorough endpoint discovery. By combining crawling and scraping, the system creates a comprehensive map of possible communication channels that are subjected to security analysis [11].

Crawling Procedure: The endpoint discovery process is as follows:

1) WSVIPER initiates Chromium browser with Playwright and generates an ensemble of randomly selected user agents for stealth.
2) The system enables navigation and monitoring by loading the page for each target URL and listening to both incoming and outgoing network traffic.
3) JavaScript files, AJAX calls, API responses, and observed traffic are used to extract patterns from WebSocket endpoints through regular expressions. The scheme used to identify a WebSocket URL is `ws` or `wss`.
4) The validation process involves validating discovered endpoints and discarding paths that are not navigable, such as fonts or images. The remaining routes are redirected for further exploration within the domain, subject to the specified request and depth limits.
5) The crawler stops when all paths have been explored or when depth limits are met.
6) The system returns a more refined set of WebSocket URLs.
7) These WebSockets are then put through a test to remove all junk that may have been entered during the process of crawling, and also remove the duplicate URLs.

By using this pipeline, it is possible to identify and capture WebSocket endpoints that are dynamically generated, delayed, or hidden, which are typically not visible to traditional crawlers.

Stage 2: Executing Vulnerability Tests on Endpoints

Our approach involved executing 90 protocol-compliant test attacks to rigorously test the security of discovered WebSocket endpoints, as per RFC 6455, the official specification for the WebSocket protocol. Fig. 2 shows the distribution of all the attacks performed in this module. Nine different types of vulnerability are present, each associated with a unique phase of WebSocket interaction or protocol handling. The classification system ensures that the tool covers a wide range of aspects while also providing sufficient depth in each category.

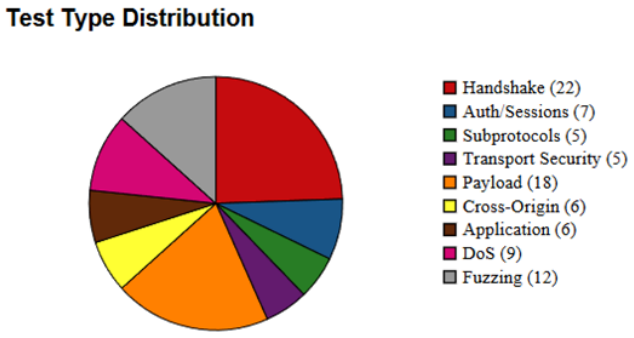

Fig. 2. Test Distribution

A. Handshake and Protocol Validation: When the WebSocket protocol is to be activated, it begins with a handshake to upgrade the HTTP/1.1 connection. This is of utmost importance as it establishes the boundary between stateless HTTP and the persistent, stateful, full-duplex communication, that is, the WebSocket.

A typical handshake request:

```
GET <path> HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
```

```
Sec-WebSocket-Key: <base64 encoded key>
Sec-WebSocket-Version: 13
```

A dependable server responds:

```
HTTP/1.1 101 Switching Protocols
Connection: Upgrade
Upgrade: websocket
Sec-WebSocket-Accept: <key>
```

Twenty-two different vulnerabilities are tested (#1–22) in this category [6], [8]. By modifying or eliminating crucial fields in the handshake request, each test endeavours to establish whether the server adheres strictly to the WebSocket upgrade specifications outlined in RFC 6455. Tests include:

- An invalid port, a non-WS scheme, and an HTTP/0.9 Handshake check to evaluate how servers handle protocol-level transgressions.
- Manipulation of headers by omitting the Sec-WebSocket-Key, duplicating the Sec-WebSocket-Key, sending the wrong upgrade header, removing host headers, and faking the host to check if servers are vulnerable to downgrade, spoofing, or misrouting.
- Encoding anomalies like Unicode URL, Long URL Path, and Case-Sensitive Headers to evaluate the robustness of URL parsing and header normalization.

Accepting non-compliant handshake requests, improper header validation, or insecure fallback mechanisms that undermine the integrity of protocol negotiation [7].

B. Authentication and Session Management Tests: This test category (#23-29) comprises seven specific tests that investigate the security measures employed by servers to manage user identity, session validation, and token access control over persistent WebSocket connections. In contrast to stateless HTTP, which mandates authentication per request, WebSocket requires a one-time handshake followed by continuous communication, creating potential risks if access control checks are not performed during the connection time.

This group simulates real-life scenarios, such as:

- No Session Cookie and Missing Authentication: Attempting unauthenticated connections to evaluate whether the server enforces login or session validation before upgrade [9], [12].
- Expired Cookie, Fake Token, and Stale Session Reconnect: Using outdated or forged credentials to test resilience against token replay and session fixation attacks.
- HTTP Session Reuse: Reusing an HTTP-authenticated session without re-validating the origin or token freshness.
- Cross-Site Cookie Hijack: Initiating connections from cross-origin contexts to exploit improperly scoped or unsecured session cookies.

Exploiting these weaknesses would result in:

- Private resources being accessed without permission.
- Session hijacking and impersonation.
- Privilege escalation.

These tests are necessary to ensure that the server maintains strict isolation between user sessions, origins, and scopes while ensuring per-connection validation is rigorous.

C. Subprotocol and Extension Handling Tests: Through the use of WebSocket, clients can request subprotocols (such as MQTT or STOMP) and extensions (like permessage-deflate) that alter communication semantics or compress the data. The category comprises five tests (#30–34) that aim at detecting improper negotiation, injection, or fallback behaviour in subprotocol and extension handling.

Test scenarios:

- Invalid Subprotocol and Unaccepted Subprotocol: Requesting unsupported or arbitrary protocol strings to determine whether the server fails closed or defaults insecurely.
- Conflicting Subprotocols: Sending multiple mutually exclusive protocols to test negotiation logic.
- Fake Extension and Conflicting Extensions: Declaring bogus or contradictory extensions to identify parser confusion, invalid negotiation, or resource overcommitment.

These attacks demonstrate if the server is at risk of:

- Protocol Downgrade Attacks: The attacker employs an inadequate or non-functional protocol.
- Unexpected Code Paths: The invalidated extension behaviour leads to unexpected code paths.
- Mismanagement of buffers as a result of incomplete or unsuccessful negotiations.

The correct execution must ensure the validity of all protocol and extension declarations, while also rejecting connections that fail during the negotiation process.

D. Transport Security and Encryption Tests: The implementation of secure WebSocket connections (wss://) over TLS, utilizing strong cryptographic primitives and providing downgrade resistance, is necessary for ensuring their confidentiality and integrity. The server's robustness to transport-layer security is evaluated in this category, which comprises five tests (#35-39).

The following vectors are examined:

- TLS Downgrade, HTTP/1.0 Downgrade, and Spoofed Connection Header: Attempting to establish connections with lower protocol versions or malformed upgrade headers to simulate downgrade attempts.
- Weak TLS Ciphers: Forcing the use of insecure or deprecated cipher suites during TLS negotiation.
- Certificate Mismatch: Initiating wss:// connections with invalid, mismatched, or self-signed certificates to test server-side certificate validation and client behaviour.

The application could be vulnerable to the following security flaws:

- Man-in-the-middle (MITM) attacks.
- Session hijacking, via interception or manipulation of traffic.
- Replay attacks that occur if encryption is implemented improperly or if it is disabled.

Well-configured servers must reject any deviation from strong TLS configuration, prevent plain-text upgrades from HTTP/1.0, and fail the connection when certificate or header anomalies are detected.

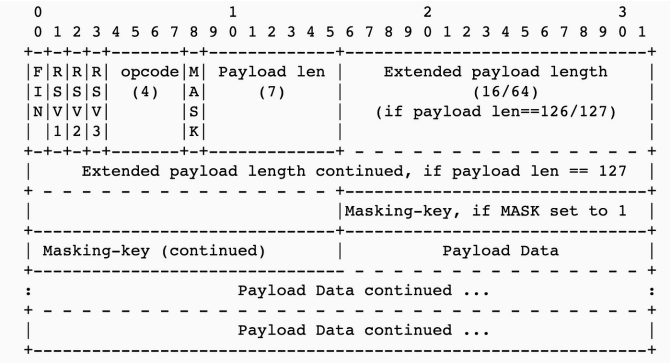

Fig. 3. WebSocket Frame Format

E. Payload Handling and Fragmentation Tests: One of the most important part of the WebSocket protocol is the framing of messages, as all messages are sent as data frames structured in the format illustrated in Fig. 3 [13]. The purpose of this test class is to determine if the server applies correct frame parsing and rejects payloads that are malformed, fragmented, or improperly masked, using 18 test cases (#40–57).

Test categories:

- Opcode Validation:
  - Sending frames with undefined or illegal opcodes is prohibited using the Undefined Opcode and the reserved Opcode.
  - Using binary as text, text as binary to exchange data types and create errors in interpretation.
- Framing and Fragmentation:
  - Zero-length fragmentation, invalid payload length, negative payload lengths, mismatched payload, and oversized control frame testing based on the implementation of fragment rules, size limits, internal tracking, etc.
  - Checking server performance during connection teardown, no connection, long closed reason, and invalid close code.
- Masking Violations:
  - To get around the masking requirement on client-to-server frames, one can use an invalid masking key to unmask a client's frame.
- Encoding and Control Frames:
  - Validating encoding and rejecting non-standard characters in textual payloads through the use of null bytes in texts.
  - The use of reserved bits to determine if servers are incorrectly supporting undocumented features or extensions can be detected by checking for invalid RSV bits.

These tests help detect:

- Vulnerabilities in parsing that could cause memory corruption or logic bypass.
- The desynchronisation which can occur due to improper fragmentation or interleaving of frames.
- Security breaches that occur due to the rules for masking or encoding that may not be strictly enforced.

When connections are encountered with incorrect or non-compliant frames, the server should immediately disconnect and record these attempts for additional scrutiny.

F. Cross-Origin and Mixed Content Tests: WebSocket applications based on browsers are vulnerable to cross-origin attacks, where the same-origin policy is a vital security measure. Attacks #58–63 are a set of six targeted tests that simulate cross-origin contexts, mixed security levels, and origin spoofing.

Test cases:

- Missing CORS Headers: Verifying if the server discloses sensitive content to unauthorized origins [6].
- Cross-Origin Iframe and PostMessage Abuse: Emulating attacks from embedded frames or malicious scripts.
- Mixed Content: Testing insecure (ws://) WebSocket connections initiated from secure (https://) web contexts.
- Missing Origin Check and Spoofed URL: Omitting or forging the Origin header to test origin-based access controls [14].

Threats:

- Cross-Site WebSocket Hijacking (CSWSH) [15], [16].
- The transmission of data to unapproved domains.
- Clickjacking and iframe abuse.
- Browsers' trust issues in HTTPS scenarios.

Our framework verifies that WebSocket endpoints perform a proper check on both the Origin and Referrer headers and reject any cross-origin attempts that are not explicitly whitelisted.

G. Application-Layer Logic and Misconfiguration Tests: This category emphasizes on exploits at the application level, rather than focusing on just basic protocol-level bugs. There are six attack cases (#64–69) that discuss inadequate sanitization, insecure headers, and overexposed API design patterns.

Key scenarios:

- Error Message Leak and Server Disclosure: Inducing errors to determine if internal server paths, debug logs, or stack traces are exposed.
- URL Path Traversal: Simulating directory escape attempts such as `../../etc/passwd`.
- Invalid Content-Type and Missing Security Headers: Checking for lax content negotiation and absent headers like Content-Security-Policy or X-Frame-Options.
- Query Parameter Flood: Sending excessive query parameters to evaluate request handling limits.

Lack of input validation, verbal error reporting, or internal routing logic is a common cause of these vulnerabilities. The

implementation of a secure WebSocket involves fail-safe error handling and sanitized responses, particularly when protocol upgrades bridge the client's direct input to backend logic.

H. DoS and Resource Management Tests: To resist resource usage by malicious actors, WebSocket servers must be kept to a minimum. We implement nine focused tests (#70–78) to test the server's ability to handle denial-of-service (DoS) conditions and resource mismanagement [17].

Tests include:

- Connection Flood and Max Connections: Stressing concurrent connections beyond expected thresholds.
- Oversized Message, Large Payload Resource Leak, and High Compression Ratio: Sending excessive or highly compressible data to evaluate memory usage.
- Idle Timeout Abuse and No Timeout Policy: Holding idle connections open indefinitely.
- TCP Half-Open Resource Leak: Simulating clients that initiate but never complete handshakes, causing resource starvation.
- No Compression Negotiation: Forcing compression even when the server has not agreed to it

The tests replicate practical DoS vectors, such as Slowloris attacks, compression bomb amplification, and connection pool exhaustion. These tests are not static. The failure of servers to disconnect idle or non-compliant clients can lead to poor performance and full-service outages.

I. Protocol Fuzzing Tests: By transmitting syntactically or semantically malformed WebSocket frames, protocol fuzzing is an advanced technique for identifying zero-day vulnerabilities, memory safety issues, and unexpected behaviours. This suite comprises 12 specialized fuzzing tests (#79-90) that examine control-plane and application-layer attack surfaces, and are described in Table 1 [18], [19].

TABLE I
FUZZING TESTS AND THEIR DESCRIPTION

Fuzz Test	Description
Malformed JSON	Incomplete or corrupted JSON string
XSS Attempt	HTML injection with `<script>` tag
Large Payload for DoS	JSON body with 1 million `A` characters
Invalid Binary Frame	Arbitrary invalid binary payload
Command Injection Simulation	Payload attempting `whoami` execution
SQL Injection Simulation	SQL logic bypass pattern (`' OR '1'='1`)
Expression Evaluation	Template-based payload: `${7*7}`
Null Bytes in JSON	JSON with embedded `\0` characters
Unicode Characters	Payload containing emoji: `[emoji symbols]`
Oversized DoS Message	2 million-character JSON message
Path Traversal Simulation	`{"path": "/../../etc/passwd"}`
PostMessage Abuse	Script injection into `window.postMessage()`

These assessments can aid in identifying:

- Unhandled exceptions, segmentation faults or crashes affecting the parser.
- Business logic bypass upon failure of input validation.
- Encoding or escaping defects that lead to XSS, command injection, or data leakage.

All fuzzing inputs are subject to WebSocket framing rules, meaning that crashes arise due to payload logic and not protocol violations. A server's ability to handle malformed inputs is a reliable indicator of its readiness for production.

V. USE CASE

This testing tool is a relatively simple tool to operate. No prior knowledge is needed to utilize it. It runs entirely on Python and only requires a couple of Python modules to be installed for successful execution. It is compatible with real-world web applications and complies with RFC 6455 [1]. It features a simple command-line interface that allows users to enter the website name. It is able to successfully identify multiple WebSocket endpoints through crawling. It provides exhaustive test coverage, which is custom-built for WebSockets. WSVIPER also provides a simple, easy-to-read PDF report that offers analysis for each website tested [20]. It is limited due to its beginner-friendly approach, as it does not provide integration with browser cookies and also does not discuss testing attacks that require integration with external tools. Table 2 compares our tool against tools in the industry, and shows that WSVIPER stands out to be a better candidate for WebSocket analysis.

TABLE II
COMPARISON OF WEBSOCKET SECURITY TESTING TOOLS

Metrics	WSVIPER	Burp Suite Pro	OWASP ZAP	WS-Attacker
Accessibility	Open Source	Closed Source	Partially Open	Limited Open
Input Methods	CSV + User Input	User Input	User Input	One URL Limit
Crawling	Exhaustive with Full JS Support	Partial JS Support	Limited Crawling	No Crawling
WebSocket Test Coverage	90 Tests	Basic Tests	Basic Tests	10 Attacks Only
Report Output	PDF + Charts	Manual Review	HTML Format	None Available

WSVIPER Link

WSVIPER is an open source tool that can be downloaded and run by visitng the following link: (WSVIPER Tool Link)

VI. RESULTS AND ANALYSIS

To demonstrate the tool in action, we built a simple demo website with some vulnerabilities, which we aim to capture through the use of WSVIPER. To illustrate the security posture of each WebSocket and identify common trends in vulnerabilities, we use a heatmap(Fig. 4) and barcharts(Fig. 5). In the heatmap, each column represents a distinct attack and each row is a unique WebSocket. This diagram allows security team to notice patterns or trends and also quickly find which vulnerabilities are affecting the endpoints. The barchart

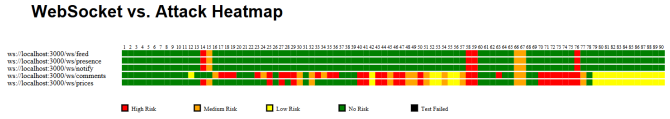

Fig. 4. Heatmap of Website vs Test Attacks

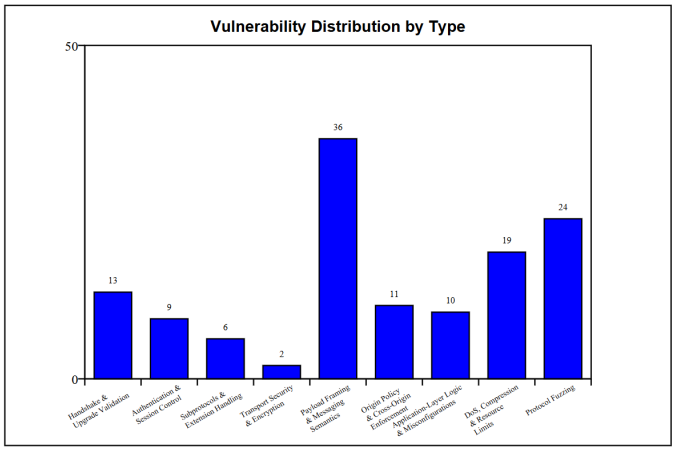

Fig. 5. Vulnerability Category vs Number of Vulnerabilities

displays the vulnerability categories against the total number of vulnerabilities found for each category. Analysing the barchart will provide an insight on the trend across endpoints and attacks and find out the type of attacks the WebSockets are more vulnerable against, allowing the security team to work efficiently. This demo website is largely secure, showing no vulnerabilities to protocol fuzzing, but has more issues with payload framing which need to be addressed. Thus the figures allow the developer to understand trends and identify issues easily and take necessary action.

VII. Conclusion

This research offers a comprehensive and automated system for identifying security flaws in WebSocket implementations across real-world web platforms. WebSocket security is often overlooked by developers, who prioritize instant communication and application-level handling over following protocol standards. There appears to be a lack of attention to the proper security of WebSockets, and this study aims to address that [6], [9]. These are fundamental aspects that need to be addressed to prevent any major security issues from arising.

In summary, WSVIPER offers the following:

- A scalable, reproducible, and automated approach to testing WebSocket security.
- A visual understanding of potentially vulnerable domains.
- Key insights for developers, security teams, and researchers to enhance the safety of applications.

References

[1] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF, RFC 6455, Dec. 2011. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6455

[2] P. Murley, Z. Ma, J. Mason, M.Bailey, and A.Kharraz, "WebSocket Adoption and the Landscape of the Real-Time Web," in *Proc. 2021 Web Conf. (WWW)*, 2021. [Online]. Available: https://faculty.cc.gatech.edu/~mbailey/publications/www21_websocket.pdf

[3] N. Nadeem, Y. Raza, A. Sajid, H. Razzaq, and S. Vidanagamachchi, "Review Analysis of WebSocket Security: Case Study," *IETI Transactions on Data Analysis and Forecasting (iTDAF)*, 2024. [Online]. Available: https://doi.org/10.3991/itdaf.v2i2.51015

[4] S. Arora, "WebSockets and Real-Time Communication," *Insights2Techinfo*. [Online]. Available: https://insights2techinfo.com/websockets-and-real-time-communication/

[5] Y. Fu and M. García-Valls, "Security aspects of full-duplex web interactions and WebSockets," *2023 20th ACS/IEEE Int. Conf. on Computer Systems and Applications (AICCSA)*, Giza, Egypt, 2023, pp. 1–8, doi: 10.1109/AICCSA59173.2023.10479302. [Online]. Available: https://ieeexplore.ieee.org/document/10479302

[6] H. Kuosmanen, "Security Testing of WebSockets," B.S. thesis, Jyväskylä Univ. of Applied Sciences, 2016. [Online]. Available: https://janet.finna.fi/Record/theseus_jamk.10024_113390?sid=5087676388

[7] BrightSec, "WebSocket Security: Top 8 Vulnerabilities and How to Solve Them," 2021. [Online]. Available: https://brightsec.com/blog/websocket-security-top-vulnerabilities

[8] K. S. Greasley and P. G. Lubbers, "WebSocket Security," in *The Definitive Guide to HTML5 WebSocket*, Berkeley, CA: Apress, 2013, pp. 129–149. [Online]. Available: https://link.springer.com/chapter/10.1007/978-1-4302-4741-8_7

[9] S. Ghasemshirazi and P. Heydarabadi, "Exploring the attack surface of WebSocket," *arXiv preprint* arXiv:2104.05324, Apr. 2021. [Online]. Available: https://arxiv.org/abs/2104.05324

[10] Microsoft, "Playwright: Fast and reliable end-to-end testing for modern web apps," GitHub, 2023. [Online]. Available: https://playwright.dev/

[11] R. Koch, *On WebSockets in penetration testing* [Diploma Thesis], Technische Universität Wien, 2013. [Online]. Available: https://resolver.obvsg.at/urn:nbn:at:at-ubtuw:1-58898

[12] B. Pandey and P. Kumar, "Security risks and best practices for securing WebSocket communications," *Int. J. Novel Res. Dev. (IJNRD)*, vol. 9, no. 7, pp. C753–C755, Jul. 2024. [Online]. Available: https://ijnrd.org/papers/IJNRD2407274.pdf

[13] K. Peacock, "WebSocket Framing, Masking, Fragmentation, and More," openmymind.net, Jul. 2013. [Online]. Available: https://www.openmymind.net/WebSocket-Framing-Masking-Fragmentation-and-More/

[14] MITRE, "CWE-1385: Missing Origin Validation in WebSockets," Common Weakness Enumeration, 2023. [Online]. Available: https://cwe.mitre.org/data/definitions/1385.html

[15] PortSwigger Web Security Academy, "Cross-site WebSocket Hijacking (CSWSH)," [Online]. Available: https://portswigger.net/web-security/websockets/cross-site-websocket-hijacking

[16] J. Somerville, "Cross-site WebSocket Hijacking (CSWSH)," Praetorian Labs Blog, May 7, 2019. [Online]. Available: https://www.praetorian.com/blog/cross-site-websocket-hijacking-cswsh

[17] J. Erkkilä, "Websocket security analysis," Aalto Univ. Sch. Sci., 2012. [Online]. Available: https://juerkkil.iki.fi/files/websocket2012.pdf

[18] I. P. A. Dharmaadi, E. Athanasopoulos, and F. Turkmen, "Fuzzing Frameworks for Server-Side Web Applications: A Survey," arXiv preprint arXiv:2406.03208, Jun. 2024. [Online]. Available: https://arxiv.org/abs/2406.03208

[19] X. Zhang et al., "A Survey on the Development of Network Protocol Fuzzing Techniques," *Electronics*, vol. 12, no. 13, p.2904, 2023. [Online]. Available: https://doi.org/10.3390/electronics12132904

[20] T. Wirasingha and N. Ruwan Dissanayake, "A Survey of WebSocket Development Techniques and Technologies," in *Proc. Professional Integration for Secure Nation*, Sep. 2016. [Online]. Available: https://www.researchgate.net/publication/309462392_A_Survey_of_WebSocket_Development_Techniques_and_Technologies